

Servidor de Aplicaciones Bajo Estrés: Monitoreo y Gestión de Recursos en un Entorno Docker sobre Ubuntu WSL2

Integrantes del Proyecto

1. Juan Stevan Cruz Diaz – 202459437
2. Jhoan Fabricio Hurtado - 202459472

Abstract

Este artículo presenta el diseño, implementación y evaluación de un entorno de servidor multicapa desplegado mediante Docker sobre Ubuntu WSL2. El entorno estuvo compuesto por un servidor web desarrollado con Next.js, una base de datos PostgreSQL, un panel de administración PgAdmin y un entorno de análisis de datos JupyterLab. Como parte del proyecto se desarrolló una aplicación web capaz de generar carga controlada mediante mecanismos de HTTP Flood, Query Flood e Insert Flood, con el propósito de provocar situaciones de sobrecarga sobre los recursos del sistema. Posteriormente, se ejecutó de forma simultánea un notebook de entrenamiento de inteligencia artificial con el fin de incrementar el consumo de CPU y memoria RAM. Durante las pruebas se utilizaron herramientas de monitoreo como htop, vmstat, docker stats y pg_stat_activity para identificar procesos responsables de la saturación, analizar el comportamiento del sistema y evaluar posibles estrategias de optimización. Los resultados obtenidos permitieron evidenciar la relación entre la carga generada, el uso de recursos y el rendimiento general del servidor, proporcionando una experiencia práctica sobre la administración de sistemas operativos en escenarios reales.

I. Introducción

Los sistemas operativos modernos tienen como responsabilidad principal administrar eficientemente los recursos de hardware disponibles. En entornos de producción es frecuente que múltiples aplicaciones compitan simultáneamente por CPU, memoria, almacenamiento y red. Cuando la demanda supera la capacidad disponible, aparecen problemas de rendimiento que afectan la calidad del servicio.

El presente trabajo tuvo como objetivo analizar el comportamiento de un servidor bajo condiciones de alta carga utilizando tecnologías ampliamente empleadas en la industria. Se construyó una arquitectura basada en contenedores Docker que permitió aislar servicios y reproducir escenarios similares a los encontrados en ambientes empresariales. La práctica estuvo orientada a identificar procesos responsables de la sobrecarga, interpretar métricas

del sistema operativo y aplicar estrategias de optimización fundamentadas en evidencia cuantitativa.

II. Marco Teórico

La gestión de procesos constituye uno de los componentes fundamentales de cualquier sistema operativo. Cada proceso requiere tiempo de CPU, espacio en memoria y acceso a dispositivos de entrada y salida. Cuando varios procesos compiten simultáneamente por estos recursos se genera contención, lo que puede producir degradación del rendimiento.

Docker es una plataforma de virtualización ligera que permite ejecutar aplicaciones en contenedores aislados compartiendo el mismo núcleo del sistema operativo. Esta tecnología facilita el despliegue de arquitecturas multicapa manteniendo independencia entre servicios.

PostgreSQL es un sistema gestor de bases de datos relacional ampliamente utilizado por su estabilidad y rendimiento. Su vista `pg_stat_activity` permite monitorear conexiones activas, consultas en ejecución y posibles bloqueos.

Por otra parte, herramientas como `htop`, `vmstat` e `iostat` permiten observar el estado interno del sistema operativo. Estas utilidades proporcionan información relacionada con uso de CPU, memoria, intercambio de memoria (swap), operaciones de entrada/salida y carga global del sistema.

Finalmente, el entrenamiento de modelos de inteligencia artificial representa una carga computacional intensiva debido al procesamiento matemático requerido durante la fase de retropropagación, convirtiéndose en una fuente importante de consumo de CPU y memoria.

III. Metodología

El entorno experimental fue desplegado sobre Ubuntu WSL2 utilizando Docker Compose. La arquitectura estuvo conformada por cinco servicios principales: servidor web Next.js, PostgreSQL, PgAdmin, JupyterLab y una aplicación generadora de carga.

La aplicación desarrollada permitió activar diferentes mecanismos de estrés mediante una interfaz web. El mecanismo HTTP Flood generó múltiples solicitudes concurrentes hacia el servidor web con el objetivo de incrementar el uso de CPU del proceso Node.js. El mecanismo Query Flood ejecutó consultas complejas sobre PostgreSQL para aumentar el consumo de CPU del motor de base de datos. El mecanismo Insert Flood realizó inserciones masivas con el propósito de incrementar la actividad de escritura sobre disco.

Posteriormente se ejecutó un notebook de inteligencia artificial encargado de realizar operaciones de entrenamiento intensivas. La ejecución simultánea de ambas cargas

permitió reproducir escenarios de saturación realistas.

Durante las pruebas se utilizaron las siguientes herramientas:

- htop: monitoreo interactivo de procesos.
- vmstat: observación de memoria virtual y actividad del sistema.
- docker stats: monitoreo de consumo por contenedor.
- pg_stat_activity: monitoreo de conexiones y consultas PostgreSQL.

Las evidencias fueron registradas mediante capturas de pantalla y archivos de salida generados durante la ejecución.

IV. Resultados

En estado de reposo el sistema presentó un consumo moderado de recursos. Los contenedores permanecieron activos sin generar una carga significativa sobre el procesador ni la memoria.

Al activar HTTP Flood se observó un incremento importante del consumo de CPU asociado al proceso Node.js. En htop el proceso apareció entre los primeros lugares de utilización, evidenciando la carga generada por las solicitudes concurrentes.

Durante Query Flood se observó un aumento en la actividad del proceso PostgreSQL. Las consultas complejas generaron múltiples conexiones activas visibles en pg_stat_activity. Además, docker stats mostró un incremento en el consumo de recursos por parte del contenedor de base de datos.

La ejecución de Insert Flood produjo un incremento en las operaciones de escritura. Este comportamiento se reflejó en el uso intensivo del subsistema de almacenamiento y en la actividad de PostgreSQL.

Cuando se ejecutó simultáneamente el notebook de inteligencia artificial, el proceso Python comenzó a consumir una cantidad considerable de CPU y memoria RAM. La coexistencia de Node.js, PostgreSQL y Python generó contención de recursos y elevó significativamente la carga global del sistema.

Los indicadores observados durante la ejecución simultánea incluyeron:

- Incremento del load average.
- Mayor tiempo de respuesta de la aplicación.
- Incremento de conexiones activas en PostgreSQL.
- Aumento sostenido del uso de memoria RAM.
- Competencia entre procesos por tiempo de CPU.

Estos resultados permitieron identificar claramente la relación entre los mecanismos de estrés implementados y los recursos afectados.

V. Análisis

Los resultados obtenidos evidenciaron que el principal cuello de botella durante las pruebas fue el procesador. La ejecución concurrente de múltiples solicitudes HTTP, consultas complejas a la base de datos y operaciones de entrenamiento neuronal produjo una competencia directa por los núcleos disponibles.

El monitoreo mediante htop permitió asociar cada incremento de consumo con el proceso correspondiente. Node.js fue responsable de gran parte de la carga web, PostgreSQL concentró el procesamiento relacionado con consultas e inserciones masivas, mientras que Python dominó el consumo asociado al entrenamiento del modelo.

La memoria RAM también mostró incrementos importantes debido a la carga generada por el notebook de inteligencia artificial. En equipos con recursos limitados este comportamiento podría conducir a intercambio de memoria (swap) o incluso a la activación del mecanismo Out Of Memory Killer.

Las herramientas de monitoreo utilizadas resultaron fundamentales para establecer la cadena causal propuesta en el proyecto: acción ejecutada, proceso afectado, recurso saturado, métrica observable y decisión de gestión.

Como estrategia de optimización se identificaron acciones como la reducción de prioridad mediante nice, limitación de CPU para contenedores específicos y reducción de parámetros de entrenamiento del modelo de inteligencia artificial. Estas medidas permiten distribuir de manera más equilibrada los recursos disponibles y mejorar la estabilidad general del sistema.

VI. Conclusiones

La práctica permitió aplicar conceptos fundamentales de administración de sistemas operativos en un entorno realista basado en contenedores Docker.

La implementación de una aplicación generadora de carga facilitó la creación de escenarios controlados de sobrecarga, permitiendo observar directamente el comportamiento del sistema frente a diferentes tipos de demanda.

Las herramientas de monitoreo proporcionaron información valiosa para identificar procesos responsables de la saturación y comprender la interacción entre CPU, memoria, almacenamiento y base de datos.

La ejecución simultánea de cargas web y procesos de inteligencia artificial reprodujo situaciones similares a las encontradas en ambientes productivos, fortaleciendo la capacidad de análisis y toma de decisiones fundamentadas.

Referencias

[1] Docker Inc., Docker Documentation, <https://docs.docker.com>

[2] PostgreSQL Global Development Group, PostgreSQL Documentation, <https://www.postgresql.org/docs>

[3] Next.js Documentation, <https://nextjs.org/docs>

[4] Ubuntu Linux Manual Pages: `htop`, `vmstat` e `iostat`.

[5] PyTorch Documentation, <https://pytorch.org/docs>